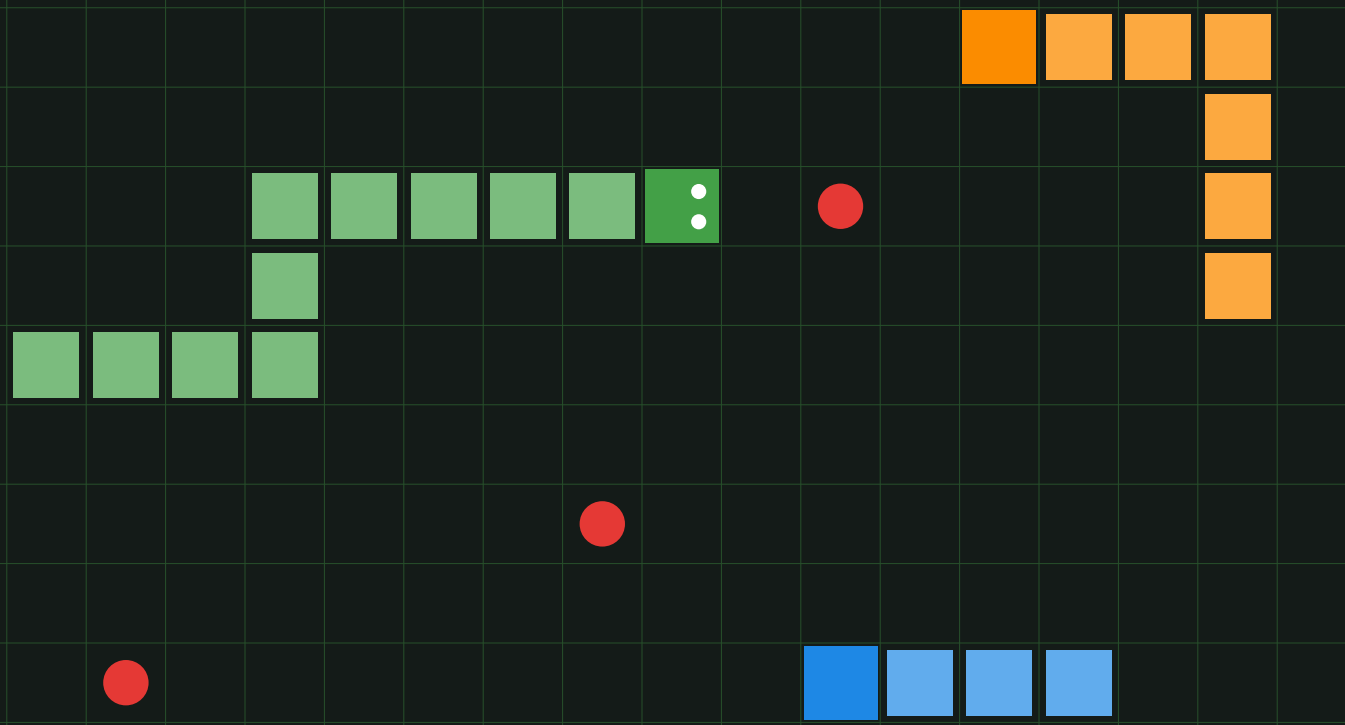
HACETTEPE UNIVERSITY
COMPUTER ENGINEERING

BBM203
FALL 2026

PROGRAMMING ASSIGNMENT 1

SNAKE ARENA



PRESS START TO BEGIN

LINKED LISTS · DOUBLY LINKED LISTS · MATRICES · DYNAMIC MEMORY · MOVE SEMANTICS

PROGRAMMING ASSIGNMENT 1

Snake Arena

Topics: Linked Lists, Doubly Linked Lists, Matrices, Dynamic Memory Allocation, Move Semantics, File I/O

LANGUAGE	C++11
DUE DATE	06/11/2026 (23:59:59)
SUBMIT	https://submit.cs.hacettepe.edu.tr

Introduction

H.U. Game Studio (**HUGS**) is building *Snake Arena*, a multiplayer version of the classic snake game where many snakes share one board, eat food, grow, bite each other and fight for the top of the leaderboard. Before the graphics team can start, the studio needs the **game engine**: a command-driven simulation that keeps track of the board, every snake and the scores.

You are the engine programmers. The engine must be fast and memory-safe, so every data structure is written by hand: the board is a dynamically allocated 2D matrix, each snake is a doubly linked list of body segments, the players form a linked list that decides the turn order, and the leaderboard is a sorted linked list.

YOUR MISSION

- keep a dynamic 2D board (walls, food and snakes) in sync with the snakes' linked lists,
- move every snake one cell per *tick*, in turn order,
- resolve eating, wall hits, self bites, bites between snakes and head-on collisions,
- keep a leaderboard that is always sorted, and
- manage all dynamic memory without leaks.

HOW TO READ THIS DOCUMENT

Level 1 tells the story of the game: what the board, the snakes and the rules are. **Level 2** describes every class, every member variable and every member function. **Level 3** explains the input file and lists every output message and when it is printed. **Level 4** describes every command one by one: its parameters, what it does and exactly which messages it prints.

LEVEL 1

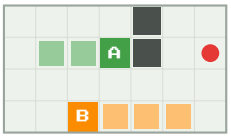
The Game

1.1 The Board

The board is a rectangular grid of cells with `rows` rows and `cols` columns. Rows are numbered from 0 (top) to `rows-1` (bottom) and columns from 0 (left) to `cols-1` (right). A cell is written as `(row,col)`. Every cell holds exactly one character:

Char	Meaning
`.`	empty cell
`#`	wall
`*`	food
`A–Z`	head of the snake with that id
`a–z`	body of the snake with that id

```
....#..
.aaA#.*
.......
..Bbbb.
```



The board does not exist until it is created with `INIT_BOARD`. Walls and food are placed with `WALL` and `FOOD`; they never move. Food disappears only when a snake eats it.

1.2 Snakes

A snake has an **id**, a single uppercase letter `A–Z`. Its body is a chain of neighbouring cells: the first cell is its **head** (shown with the uppercase id), the others are its body (shown with the lowercase id). The last cell is its **tail** and the second cell is its **neck**. The number of cells is the snake's **length** (at least 1).

Every snake has a **direction**: `UP`, `DOWN`, `LEFT` or `RIGHT`. Directions are **absolute** (they refer to the map, not to the snake): `UP` always means towards row 0, `DOWN` towards larger row numbers, `LEFT` towards column 0 and `RIGHT` towards larger column numbers, whichever way the snake is facing. Every snake also has a **score** that starts at 0 and grows by 1 each time it eats.

A snake is created with `SPAWN`. Its head is placed on the given cell and its body extends *behind* it, that is, in the direction opposite to its direction. A snake can change its direction with `TURN` (but it may never turn onto its own neck) or turn around completely with `REVERSE` (its tail becomes its head).

1.3 Players and Turn Order

All snakes in the game form the **player list**. A new snake is added to the end of the list, so the list is always in spawn order. This order is the **turn order**: in every tick the snakes move one after the other in this order. A dead snake is removed from the list.

1.4 Time: Ticks

The game runs in **ticks**, counted by the *tick counter* (`tickCount`, a member of `Arena`, starting from 0). In one tick:

1. the tick counter is increased by one,
2. every snake that is alive moves **one cell** in its direction, one snake after the other, in turn order. Moves are sequential: each snake sees the board exactly as the snakes before it left it in this tick,
3. a snake that dies during the tick (even before its own move) does not move,
4. at the end of the tick, the dead snakes are removed from the player list and deleted.

1.5 Moving Into a Cell

When a snake moves, its head goes into the cell in front of it. Into an **empty** cell (or its own tail, which moves away at the same time) it simply moves: its length stays the same. Into **food** it eats: it grows by one and its score increases by one. Into a **wall** or out of the board it dies, and into its own body it bites itself and dies. Into **another snake's body** it bites that snake: the bitten snake loses that cell and everything behind it (they become food), and the biter eats. Into **another snake's head** it is a head-on collision: the longer snake wins, and with equal lengths both die. The exact rules and messages are in Section 4.6.

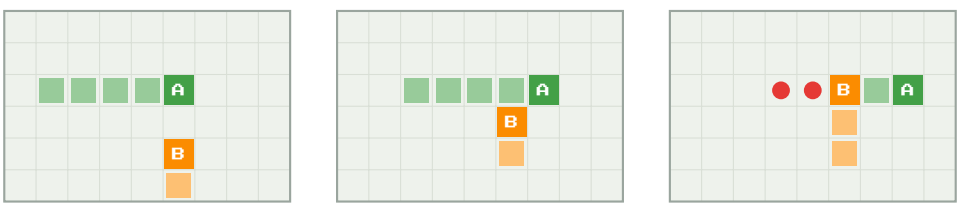
1.6 When a Snake Dies

A snake dies only during a tick (hitting a wall, biting itself or losing a head-on collision). When a snake dies, **every** cell of it, including its head, becomes food. It is removed from the leaderboard immediately, it does not move any more, and it is removed from the player list and deleted at the end of the tick. After that, its id is free again and a new snake with the same id may be spawned.

1.7 The Leaderboard

The leaderboard lists the snakes that are alive, best first. It is always sorted by **(1)** higher score first and **(2)** on equal scores, the smaller id first (`A` before `B`). A new snake enters the leaderboard when it is spawned, a snake moves to its new place every time its score changes, and a snake leaves the leaderboard when it dies.

1.8 Example: A Bite



START AFTER TICK 1 AFTER TICK 2

In tick 2, A moves first: its head goes to `(2,7)` and its body is now `(2,6)…(2,3)`. Then B moves up into `(2,5)`, which is A's third cell. A is cut there and loses `(2,5)`, `(2,4)` and `(2,3)`, which become food. B eats the food at `(2,5)`; the two other food cells stay on the board. The input and the output of this example are in Section 4.6.

LEVEL 2

The Classes

2.1 Overview

File	Contents
`Enums.h/.cpp`	`Direction` and helper functions. GIVEN
`Segment.h/.cpp`	One cell of a snake: a node of a doubly linked list. GIVEN
`Board.h/.cpp`	The board (grid): a dynamic 2D `char` matrix.
`SnakeBody.h/.cpp`	The body of a snake: a doubly linked list of `Segments`.
`Snake.h/.cpp`	One snake: id, direction, score, body; also a node of the player list.
`Leaderboard.h/.cpp`	The leaderboard: a sorted singly linked list of `RankNodes`.
`Arena.h/.cpp`	The controller: owns everything and executes the commands.
`main.cpp`	Reads the input file and passes each line to `Arena`. GIVEN

In the tables below, GIVEN marks code that is already written in the starter code and YOU marks what you implement. Members without a tag are defined inline in the header (already written). Do not change any header file (Section 5).

> **WHO OWNS WHAT**
>
> `Arena` owns the `Board`, the `Leaderboard` and every `Snake` (snakes are created with `new` by `SPAWN`). A `Snake` owns its `SnakeBody`, and a `SnakeBody` owns its `Segments` (created with `new`). A `RankNode` only *points* to a snake: the leaderboard deletes its nodes but **never** the snakes. Whoever owns an object deletes it: segments are deleted when they are popped, cut off or cleared; a snake is deleted by `Arena` when it is removed at the end of a tick, when `INIT_BOARD` starts a new game, or when the `Arena` object is destroyed at the end of the program.

2.2 Direction and Segment

These two are GIVEN completely.

Member	Specification
`enum class Direction`	`UP`, `DOWN`, `LEFT`, `RIGHT`.
`directionToString(d)`, `parseDirection(s, out)`	Direction ↔ text. `parseDirection` accepts any letter case and returns `false` for a non-direction.
`oppositeDirection(d)`, `rowDelta(d)`, `colDelta(d)`, `toUpper(s)`	The opposite direction; the row / column change of one step (`UP` is $(-1, 0)$, `RIGHT` is $(0, +1)$); an uppercase copy.
`class Segment`	One cell of a snake's body, i.e. one node of the `SnakeBody` list. It stores its cell (`row`, `col`) and two pointers: `prev` to the neighbouring segment closer to the head and `next` to the one closer to the tail (`nullptr` at the ends). It has get/set functions for these and is printed as `(row,col)`. `SnakeBody` is declared a `friend`, so `SnakeBody`'s functions may use these private members directly.

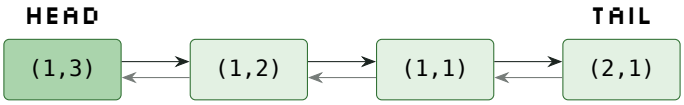
2.3 Board

Members: `rows`, `cols` (size, 0 if uninitialized) and `char **cells` (`cells[r][c]` is cell `(r,c)`, `nullptr` if uninitialized). The size is known only at run time, so the matrix is allocated dynamically. A board cannot be copied but can be **moved**; `Arena` creates a new board with `board = Board(r, c);`.

Member	Specification
`Board()`	Uninitialized board (0, 0, `nullptr`). GIVEN
`Board(int rows, int cols)`	A `rows`×`cols` matrix of `EMPTY_CELL`. YOU
`~Board()`	Frees the matrix; must also work for an uninitialized or moved-from board. YOU
`Board(Board &&other)`	Takes over `other`'s matrix; `other` becomes uninitialized. YOU
`operator=(Board &&other)`	Frees its own matrix, takes over `other`'s, leaves `other` uninitialized, returns `*this`. Self-assignment changes nothing. YOU
`bool inBounds(int r, int c)`	`true` if `(r,c)` is inside the board (any `r`, `c`, also negative). YOU
`getRows, getCols, isInitialized, get, set, print`	Size; `cells != nullptr`; read/write a cell inside the board; print (Section 3.4). GIVEN

2.4 SnakeBody

Members: `Segment *head, *tail` and `int length`. A **doubly linked list** of segments, `head` is the snake's head and `tail` its last segment.



After every operation `head`, `tail`, `length` and all `prev`/`next` links must be correct; an empty list has `head = tail = nullptr` and `length = 0`.

Copying is not allowed (`= delete`).

Member	Specification
`SnakeBody(), ~SnakeBody()`	Empty list; the destructor deletes all segments. GIVEN
`SnakeBody(SnakeBody &&other)`	Takes over `other`'s segments; `other` becomes a valid empty list. YOU
`operator=(SnakeBody &&other)`	Deletes its own segments, takes over `other`'s, leaves `other` empty, returns `*this`. Self-assignment changes nothing. YOU
`bool isEmpty()`	`true` if there are no segments. YOU
`pushFront(row, col)`	Adds a new segment at `(row,col)` as the new head. YOU
`pushBack(row, col)`	Adds a new segment at `(row,col)` as the new tail. YOU
`bool popBack(int &row, int &col)`	Removes and deletes the tail, writes its cell into `row`/`col`, returns `true`; `false` if empty. YOU
`void reverse()`	Reverses the list in place (the tail becomes the head). No segment is created or deleted. YOU
`Segment *findAt(row, col)`	The segment at `(row,col)`, or `nullptr`. YOU
`SnakeBody splitAt(row, col)`	The segment at `(row,col)` and all segments after it (towards the tail) are removed from this list and returned as a new body, in the same order. If there is no such segment, returns an empty body and changes nothing. YOU
`void clear()`	Deletes every segment; the list is empty and usable afterwards. YOU
`getHead, getTail, getLength, operator<<`	Getters; prints `(r,c) (r,c) ...` from head to tail. GIVEN

2.5 Snake

Members: `id` (`'A'–'Z'`), `direction`, `score` (starts at 0), `alive`, `SnakeBody body` (head first) and `Snake *next`, the next snake in the player list (singly linked, turn order).

Member	Specification
`Snake(char id, Direction d)`	Empty body, score 0, alive, `next = nullptr`. GIVEN
`getters, setNext, setAlive, addScore`	Read / change the members above. GIVEN
`bool turn(Direction newDir)`	Sets the direction and returns `true`, unless one step in `newDir` would put the head onto the neck (second segment): then returns `false` and changes nothing. This is not the same as "the opposite direction" (Section 1.2). Length 1: always allowed. YOU
`void reverse()`	Reverses the body. New direction: from the new second segment towards the new head; for length 1, the opposite direction. YOU
`getNextHeadPosition(int &row, int &col)`	The cell one step from the head in the current direction (may be outside the board). YOU

2.6 RankNode and Leaderboard

Members: `RankNode *head` and `int size`. A **sorted singly linked list** of `RankNodes` (`Snake *snake`, `RankNode *next`), one per alive snake: higher score first, on equal scores the smaller id first. A node points to a snake but does **not** own it.

Member	Specification
`Leaderboard(), ~Leaderboard()`	Empty list; the destructor deletes all nodes. GIVEN
`insertSorted(Snake *s)`	Adds a node for `s` at its correct place (head, middle or end). YOU
`bool remove(Snake *s)`	Deletes the node of `s` (not the snake) and returns `true`; `false` if `s` is not in the list. YOU
`reposition(Snake *s)`	After the score of `s` changed, moves its node to its correct place. YOU
`clear()`	Deletes all nodes (not the snakes); the list is usable afterwards. YOU
`getHead, isEmpty, getSize, print`	Getters; print (Section 3.4). GIVEN

2.7 Arena

`Arena` is the game itself: it holds the board, the snakes and the leaderboard and runs the commands. (The *board* is only the grid of cells, class `Board`.) **Members:** `Board board`, the player list `Snake *firstSnake, *lastSnake` (spawn order), `Leaderboard leaderboard` and `int tickCount` (the tick counter, ticks since the last `INIT_BOARD`).

Member	Specification
`Arena() / ~Arena()`	No board, no snakes GIVEN / deletes all remaining snakes YOU
`processCommand(line)`	Reads the command word and calls the right handler. GIVEN
`cmdInitBoard, cmdWall, cmdFood, cmdSpawn, cmdTurn, cmdReverse, cmdTick`	One handler per command, as described in Level 4. `cmdWall` GIVEN, the others YOU
`printSnakes()`	`PRINT_SNAKES` (Section 3.4). GIVEN
`findSnake(id)`	The alive snake with this id, or `nullptr`. YOU
`addSnake(s), clearSnakes(), removeDeadSnakes()`	Append to the player list; delete all snakes (and clear the leaderboard); delete the dead ones (others keep their order). YOU
`killSnake(s), moveSnake(s), tick()`	A snake dies / moves one cell / one tick, as in Section 4.6. YOU

KEEP THEM IN SYNC

The board and the linked lists describe the same snakes and must always agree. Whenever a snake moves, grows, is cut, reverses or dies, update both. The board answers "what is in this cell?" in $O(1)$; the linked list knows the order of the segments (which one is the tail, which one is the neck). When a snake bites another one, the board tells you *whose* body is in the target cell (the lowercase letter) and `findAt` / `splitAt` find the segment in that snake's list.

LEVEL 3

Input and Output

3.1 Input File

The program is run as `./SnakeArena <input_file>` and reads the file line by line. Every line holds one command; there are no empty lines or comment lines.

THE INPUT IS ALWAYS WELL-FORMED

You do **not** need to check the format of the commands. In every input file:

- every command word is one of the ten commands of Level 4, written in uppercase,
- every command has exactly the parameters given in Level 4, of the right type, separated by single spaces (ids are one uppercase letter, directions are `UP`, `DOWN`, `LEFT`, `RIGHT`, numbers are whole numbers),
- the first command is `INIT_BOARD` with $1 \leq$ `rows`, `cols` ≤ 100, `SPAWN` has `len` ≥ 1 and `TICK` has `n` ≥ 1.

What you **do** have to check are the situations of the game itself: a cell that is outside the board or not empty, a snake id that already exists or does not exist, and a turn onto the neck. They are listed with each command in Level 4.

3.2 Output Rules

All output goes to `std::cout`. Outputs are checked automatically, so the text, spacing and punctuation must be **exactly** as in this section: `<..>` parts are replaced by values, coordinates are printed as `(row,col)` without spaces, directions in uppercase. Every message is one line. Spaces at the end of a line and empty lines at the end of the output are ignored by the grader; everything else is compared exactly. Every line is scored on its own: a wrong, missing or extra line costs only that line. Still, remove debug prints from your classes before submitting (`main.cpp` is not used by the grader, so prints there do not affect your grade).

3.3 All Messages

These are **all** the lines your program can print (besides the print formats in 3.4).

Message	Printed when	See
`Board initialized with <r> rows and <c> columns.`	`INIT_BOARD`	4.1
`Wall placed at (<r>,<c>).`	a wall is placed	4.2
`Food placed at (<r>,<c>).`	food is placed	4.2
`Snake <id> spawned at (<r>,<c>) facing <DIR> with length <len>.`	a snake is created	4.3
`Snake <id> is now facing <DIR>.`	a turn succeeds	4.4
`Snake <id> reversed and is now facing <DIR>.`	a snake is reversed	4.5
`--- Tick <n> ---`	at the start of every tick	4.6
`Snake <id> ate food at (<r>,<c>).`	a snake eats	4.6
`Snake <id> hit a wall at (<r>,<c>).`	a snake moves into a wall or out of the board	4.6
`Snake <id> bit itself at (<r>,<c>).`	a snake moves into its own body	4.6
`Snake <id> bit Snake <id2> at (<r>,<c>).` `Snake <id2> lost <k> segment(s).`	a snake bites another one	4.6
`Snake <id> and Snake <id2> collided head-on at (<r>,<c>).`	head-on collision	4.6
`Snake <id> died with length <len> and score <score>.`	a snake dies	4.6
`Error: Cell (<r>,<c>) is not available.`	`WALL`/`FOOD` on a cell outside the board or not empty	4.2
`Error: Snake <id> already exists.`	`SPAWN` with the id of an alive snake	4.3
`Error: Cannot spawn Snake <id> at (<r>,<c>).`	a cell of the new snake is outside or not empty	4.3
`Error: Snake <id> not found.`	`TURN`/`REVERSE` for an id that is not alive	4.4, 4.5
`Error: Snake <id> cannot turn back.`	a turn onto the neck	4.4

A command prints at most one error message. After an error the command does nothing else (nothing changes).

3.4 Print Formats

The three print functions are given in the starter code:

```
--- Board 4x7 ---
....#..
.aaA#.*
.......
..Bbbb.
--- Snakes ---
A RIGHT length=3 score=0: (1,3) (1,2) (1,1)
B LEFT length=4 score=1: (3,2) (3,3) (3,4) (3,5)
--- Leaderboard ---
1. B score=1 length=4
2. A score=0 length=3
```

`PRINT_BOARD` prints the size line and then every row. `PRINT_SNAKES` lists the snakes in turn order with their cells from head to tail. `PRINT_LEADERBOARD` lists the snakes in leaderboard order. With no snakes, `PRINT_SNAKES` and `PRINT_LEADERBOARD` print `No snakes in the arena.` after their title line.

LEVEL 4

The Commands

Each command below lists what it does, its output and the errors it has to check.

4.1 INIT_BOARD <rows> <cols>

Effect: starts a new game on a new empty `rows`×`cols` board. It may be used again later. If there already is a game, it is thrown away completely: all snakes are deleted (**no** death messages are printed), the leaderboard is cleared, the old board with its walls and food is freed, and the tick counter is set back to 0.
Output: `Board initialized with <rows> rows and <cols> columns.`

4.2 WALL <r> <c> and FOOD <r> <c>

Effect: puts a wall (`WALL`) or food (`FOOD`) on cell `(r,c)`.
Output: `Wall placed at (<r>,<c>).` or `Food placed at (<r>,<c>).`
Error: `Error: Cell (<r>,<c>) is not available.` if the cell is outside the board or is not empty (wall, food or snake).

4.3 SPAWN <id> <r> <c> <dir> <len>

Effect: creates a snake with head `(r,c)`, direction `dir`, length `len`, score 0. The body extends **behind** the head, in the opposite direction of `dir`: `SPAWN A 2 3 RIGHT 3` occupies `(2,3)` (head), `(2,2)` and `(2,1)` (tail). On the board the head is the uppercase id and the other cells are the lowercase id. The snake is appended to the end of the player list and inserted into the leaderboard.
Output: `Snake <id> spawned at (<r>,<c>) facing <DIR> with length <len>.`
Errors, in this order: `Error: Snake <id> already exists.` if an alive snake has this id; then `Error: Cannot spawn Snake <id> at (<r>,<c>).` if **any** of the `len` cells is outside the board or not empty (wall, food or another snake); the coordinates are the head's and nothing is placed.

4.4 TURN <id> <dir>

Effect: changes the snake's direction with `Snake::turn` (neck rule in Section 2.5). Directions are absolute (Section 1.2). The snake does not move until the next tick.
Output: `Snake <id> is now facing <DIR>.` (also when <DIR> is the current direction)
Errors, in this order: `Error: Snake <id> not found.` if no alive snake has this id; then `Error: Snake <id> cannot turn back.` if the turn would put the head onto the neck (the direction does **not** change).

4.5 REVERSE <id>

Effect: reverses the snake with `Snake::reverse` (Section 2.5): its tail becomes its head and its direction changes. On the board the old head becomes lowercase and the new head uppercase. The snake does not move until the next tick.
Output: `Snake <id> reversed and is now facing <DIR>.` with the new direction.
Error: `Error: Snake <id> not found.`

4.6 TICK [n]

Effect: runs `n` ticks one after the other. Each tick (Section 1.4):

1. increases the tick counter and prints `--- Tick <n> ---` with the new value of the counter (the numbering continues over `TICK` commands and restarts after `INIT_BOARD`),
2. moves every alive snake once, in turn order, as in the table below (a tick in which nothing happens prints only the tick line),
3. removes and deletes the dead snakes.

Moving one snake `S` whose next head cell is `(nr,nc)`. The messages are printed in the order shown:

Cell in front	Changes and messages
Empty, or S's own tail	S moves: `(nr,nc)` becomes its head and its tail cell becomes empty (for its own tail: becomes the new head). Length and score do not change. **No message.**
Food	S eats and grows: `(nr,nc)` becomes its head and its tail stays. Length $+1$, score $+1$, the leaderboard is updated. `Snake S ate food at (nr,nc).`
Outside the board or a wall	`Snake S hit a wall at (nr,nc).` then S dies: `Snake S died with length L and score P.` For a cell outside the board the message has that outside cell, e.g. `(-1,3)` or `(2,9)` on a board with 9 columns.
S's own body (not the tail)	`Snake S bit itself at (nr,nc).` then `Snake S died with length L and score P.`
Body of snake T	T is cut at `(nr,nc)`: that cell and every cell after it (towards T's tail) are removed from T and become food. T keeps at least its head, its score and stays alive. Then S eats the food at `(nr,nc)` as in the *Food* row: `Snake S bit Snake T at (nr,nc). Snake T lost K segment(s). Snake S ate food at (nr,nc).` `K` is the number of cells cut from T (at least 1).
Head of snake T	First `Snake S and Snake T collided head-on at (nr,nc).`, then with `L(S)` and `L(T)` the lengths: ■ `L(S) > L(T)`: T dies (its death message), then S eats the food at `(nr,nc)` (`Snake S ate food at (nr,nc).`). ■ `L(S) < L(T)`: S dies (its death message). T does not change. ■ `L(S) = L(T)`: both die: first S's death message, then T's.

Death. When a snake dies, `Snake <id> died with length <len> and score <score>.` is printed right after the message of the cause, with the length and score it had at that moment. All its cells become food, it leaves the leaderboard at once and it is deleted at the end of the tick (Section 1.6). Death is not a command; it only happens inside a tick.

Example (the bite of Section 1.8):

```
INIT_BOARD 6 9
SPAWN A 2 5 RIGHT 5
SPAWN B 4 5 UP 2
TICK 2
PRINT_SNAKES
```

```
Board initialized with 6 rows and 9 columns.
Snake A spawned at (2,5) facing RIGHT with length 5.
Snake B spawned at (4,5) facing UP with length 2.
--- Tick 1 ---
--- Tick 2 ---
Snake B bit Snake A at (2,5). Snake A lost 3 segment(s).
Snake B ate food at (2,5).
--- Snakes ---
A RIGHT length=2 score=0: (2,7) (2,6)
B UP length=3 score=1: (2,5) (3,5) (4,5)
```

4.7 PRINT_BOARD, PRINT_SNAKES, PRINT_LEADERBOARD

Print the board, the snakes and the leaderboard in the formats of Section 3.4.

LEVEL 5

Implementation Notes

- You are given a starter code with all headers, `Enums`, `Segment`, the print functions and `main.cpp`. Complete every function marked `// TODO` (marked `you` in Level 2).
- If you need extra helper functions, write them as `static` functions inside your `.cpp` file (like `isSnakeId` in `Arena.cpp`). Define the methods of each class only in that class's own `.cpp` file.
- **Do not add new classes or files.** The grader compiles only `Board.cpp`, `SnakeBody.cpp`, `Snake.cpp`, `Leaderboard.cpp` and `Arena.cpp`. `main.cpp` is not compiled, so do not put functions your classes need into it.
- Every `new` must have a matching `delete` (`new[]` → `delete[]`). Who deletes what is described in *Who owns what* (Level 2): segments are deleted by their `SnakeBody`, snakes by the `Arena` (at the end of the tick in which they die, on a new `INIT_BOARD`, and in `~Arena()` at the end of the program). `Leaderboard` functions delete only `RankNodes`, never a `Snake`.
- Classes are graded separately: the grader tests each of your classes together with correct versions of the other classes. Each class must follow its own specification.

COMMON BUGS

For every list operation, test the edge cases: an empty list, a list with one element, the first and the last element. After inserting, removing or splitting, both ends of the list and its length must be correct. A moved-from object must still be safe to destroy, and assigning an object to itself must not break it.

FORBIDDEN

- **Do not modify the header files.** Do not rename classes, functions or member variables and do not add new members. The grader compiles your files with the original headers.
- **No STL containers:** `std::vector`, `std::list`, `std::map`, `std::set`, `std::queue`, `std::deque`, … All linked structures and the board matrix are implemented by hand. (`std::string` and string streams are allowed.)

LEVEL 6

Build, Run and Test

Compile all `.cpp` files together and run the program with an input file (the same commands work on Linux, macOS and Windows with MinGW; on Windows run `SnakeArena.exe`):

```
g++ -std=c++11 *.cpp -o SnakeArena
./SnakeArena inputs/t1_basic.txt
```

The sample inputs and outputs (`Sample_IO_PA1`) are only examples; your submission will be tested with different inputs. Memory errors and leaks can be checked with Valgrind on Linux (`valgrind --leak-check=full ./SnakeArena inputs/t1_basic.txt`); on other systems, use Tur3Bo, which runs the same checks.

SNAKE ARENA VIEWER

Open `SnakeArenaViewer.html` in any browser to watch an input being played tick by tick, and paste your own output to compare it with the expected one line by line.

You must test your code on Tur3Bo Grader `https://test-grader.cs.hacettepe.edu.tr/` (does not count as submission!).

Grading Policy

Item	Weight
Linked lists and snake moves (70%)	
`SnakeBody`: insertion (`pushFront`, `pushBack`)	7%
`SnakeBody`: removal (`popBack`, `clear`)	8%
`SnakeBody`: search (`findAt`)	5%
`SnakeBody`: split (`splitAt`)	7%
`SnakeBody`: reverse	7%
`SnakeBody`: move constructor / move assignment	7%
`Leaderboard`: sorted insertion	8%
`Leaderboard`: removal	6%
`Leaderboard`: reposition	4%
`Leaderboard`: clear	2%
`Snake`: reverse (body and direction)	6%
`Snake`: turn (neck rule) and next head position	3%
Memory (10%)	
No memory leaks	5%
No memory errors	5%
Other (20%)	
`Board` (dynamic matrix, move semantics)	8%
Game output: commands, movement, walls, reverse, bites, head-on, complete output	12%

How memory is checked. Your program is run with **Valgrind** (the same tool as `make mem`) on hidden inputs, and a short test program that calls your classes directly (for example moving one `SnakeBody` into another) is also run with Valgrind.

- **No memory leaks:** at the end of the program everything allocated with `new` has been deleted. Valgrind must report 0 bytes *definitely*, *indirectly* and *possibly lost*.
- **No memory errors:** while the program runs it never uses memory it must not use: no reading or writing a deleted segment or snake (*invalid read/write*), no deleting the same memory twice, no using uninitialized values. Valgrind must report `ERROR SUMMARY: 0 errors`.

They are graded separately because they are different bugs: a program can delete everything and still use a snake after deleting it, or never touch deleted memory and still forget to delete some of it. Memory points are given only to submissions that earn at least 30 points from the other tests (an empty program has no memory leaks). **STL penalty:** if a file uses a forbidden STL container, all tests of that file get 0 points (for `Arena.cpp` this is the whole game output). You need to get a **non-zero** grade for your submission to be accepted. Submissions with grade 0 will be counted as NO SUBMISSION!

Important Notes

- Do not miss the deadline: **06/11/2026 (23:59:59)**.
- Save all your work until the assignment is graded.
- The assignment solution you submit must be your original, individual work. Duplicate or similar assignments are both going to be considered as cheating.
- You can ask your questions via Piazza, and you are supposed to be aware of everything discussed on Piazza.
- You must submit your work via `https://submit.cs.hacettepe.edu.tr/` with the file hierarchy below. `Enums`, `Segment` and `main.cpp` are not submitted.

```
b<studentID>.zip
 |- Board.cpp          Board.h
 |- SnakeBody.cpp      SnakeBody.h
 |- Snake.cpp          Snake.h
 |- Leaderboard.cpp    Leaderboard.h
 |- Arena.cpp          Arena.h
```

- You MUST use the starter code.
- This file hierarchy must be zipped before submitted (not .rar, only .zip files are supported).
- Do not submit any object or executable files. Only header and C++ files are allowed.

Academic Integrity Policy

All work on assignments must be done individually. You are encouraged to discuss the given assignments with your classmates, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in

pseudocode) will not be tolerated. In short, turning in someone else's work (including work available on the internet), in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else.

SIMILARITY CHECK

The submissions will be subjected to a similarity check. Any submissions that fail the similarity check will not be graded and will be reported to the ethics committee as a case of academic integrity violation, which may result in the suspension of the involved students. Do not copy any code produced by LLM tools (e.g., ChatGPT, Gemini, Copilot) or any online source.